

=====

CAPSTONE PROJECT

NOTEBOOK 21: BATCH INFERENCE DEMO FOR 10-GENRE SYSTEM

=====

Purpose:

This notebook demonstrates the final 10-genre hybrid genre

prediction system on multiple audio files.

The goal is to:

1. Load the trained 10-genre structured model and CNN
2. Select a sample of labelled tracks from the 10-genre pilot
3. Run hybrid prediction on each audio file
4. Compare actual vs predicted genres

5. Export the results for analysis and presentation

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import joblib
import numpy as np
import pandas as pd
import librosa
import tensorflow as tf
import matplotlib.pyplot as plt

from scipy.stats import kurtosis, skew
```

In [2]:

```
# =====
# 2. LOAD SAVED MODELS, SCALER, LABELS, AND FUSION WEIGHTS
# =====

structured_model = joblib.load("../models/best_structured_model_10genre.joblib")
structured_scaler = joblib.load("../models/structured_scaler_10genre.joblib")
cnn_model =
tf.keras.models.load_model("../models/improved_audio_cnn_10genre.keras")

structured_feature_columns = np.load(
    "../data/processed/structured_feature_columns_10genre.npy",
    allow_pickle=True
)

class_names = np.load(
    "../data/processed/structured_label_classes_10genre.npy",
    allow_pickle=True
)

STRUCTURED_WEIGHT = 0.4
CNN_WEIGHT = 0.6

print("Loaded final 10-genre models.")
print("Class names:", class_names)
print("Fusion weights -> Structured:", STRUCTURED_WEIGHT, "| CNN:", CNN_WEIGHT)
```

Loaded final 10-genre models.

Class names: ['Classical' 'Electronic' 'Experimental' 'Folk' 'Hip-Hop' 'Instrumental' 'International' 'Pop' 'Rock' 'Spoken']

Fusion weights -> Structured: 0.4 | CNN: 0.6

In [3]:

```
# =====
# 3. LOAD AND CLEAN 10-GENRE PILOT METADATA
# =====

pilot_df = pd.read_csv("../data/processed/audio_large_pilot_metadata_10genre.csv")

print("Original pilot shape:", pilot_df.shape)
display(pilot_df.head())

print("\nOriginal class distribution:")
print(pilot_df["genre_top"].value_counts())

print("\nOriginal split distribution:")
print(pilot_df["split"].value_counts())

def can_load_audio(file_path, sr=22050, duration=3):
    try:
        y, _ = librosa.load(file_path, sr=sr, duration=duration)
        return y is not None and len(y) > 0
    except:
        return False

pilot_df["can_load"] = pilot_df["audio_path"].apply(can_load_audio)

print("\nReadable audio files:")
print(pilot_df["can_load"].value_counts())

pilot_df = pilot_df[pilot_df["can_load"] == True].copy().reset_index(drop=True)

print("\nClean pilot shape:", pilot_df.shape)

print("\nClean class distribution:")
print(pilot_df["genre_top"].value_counts())

print("\nClean split distribution:")
print(pilot_df["split"].value_counts())
```

Original pilot shape: (6329, 5)

	track_id	genre_top	split	audio_path	audio_exists
0	183	Rock	test	../data/raw/audio/fma_large\000\000183.mp3	True
1	184	Rock	test	../data/raw/audio/fma_large\000\000184.mp3	True
2	191	Folk	training	../data/raw/audio/fma_large\000\000191.mp3	True
3	205	Folk	training	../data/raw/audio/fma_large\000\000205.mp3	True
4	306	Rock	training	../data/raw/audio/fma_large\000\000306.mp3	True

Original class distribution:

```
genre_top
Experimental    1781
Rock            1546
Electronic      626
Pop             456
Folk            404
Hip-Hop         382
Instrumental     375
Spoken          301
International    241
Classical       217
```

Name: count, dtype: int64

Original split distribution:

```
split
test          2360
validation    2169
training      1800
```

Name: count, dtype: int64

C:\Users\jdevo\AppData\Local\Temp\ipykernel_34200\2870184446.py:18: UserWarning: PySoundFile failed. Trying audioread instead.

```
y, _ = librosa.load(file_path, sr=sr, duration=duration)
```

c:\Users\jdevo\AppData\Local\Programs\Python\Python39\lib\site-packages\librosa\core\audio.py:184: FutureWarning:

```
librosa.core.audio.__audioread_load
```

Deprecated as of librosa version 0.10.0.

It will be removed in librosa version 1.0.

```
y, sr_native = __audioread_load(path, offset, duration, dtype)
```

Readable audio files:

can_load

True 6312

False 17

Name: count, dtype: int64

Clean pilot shape: (6312, 6)

Clean class distribution:

genre_top

Experimental 1779

Rock 1545

Electronic 626

Pop 456

Folk 404

Hip-Hop 382

Instrumental 374

Spoken 299

International 230

Classical 217

Name: count, dtype: int64

Clean split distribution:

split

test 2356

validation 2167

training 1789

Name: count, dtype: int64

In [4]:

```
# =====
# 4. HELPER FUNCTIONS
# =====

def safe_kurtosis(x):
    try:
        value = kurtosis(x, fisher=True, bias=False, nan_policy="omit")
        if np.isnan(value):
            return 0.0
        return float(value)
    except:
        return 0.0

def safe_skew(x):
    try:
        value = skew(x, bias=False, nan_policy="omit")
        if np.isnan(value):
            return 0.0
        return float(value)
    except:
        return 0.0

def compute_summary_stats(feature_matrix, feature_name):
    if feature_matrix.ndim == 1:
        feature_matrix = feature_matrix.reshape(1, -1)
```

```

summary = {}

for i in range(feature_matrix.shape[0]):
    values = np.asarray(feature_matrix[i], dtype=float)

    summary[f"{feature_name}_kurtosis_{i+1:02d}"] = safe_kurtosis(values)
    summary[f"{feature_name}_max_{i+1:02d}"] = float(np.nanmax(values))
    summary[f"{feature_name}_mean_{i+1:02d}"] = float(np.nanmean(values))
    summary[f"{feature_name}_median_{i+1:02d}"] = float(np.nanmedian(values))
    summary[f"{feature_name}_min_{i+1:02d}"] = float(np.nanmin(values))
    summary[f"{feature_name}_skew_{i+1:02d}"] = safe_skew(values)
    summary[f"{feature_name}_std_{i+1:02d}"] = float(np.nanstd(values))

return summary

def extract_structured_features(audio_path, sr=22050, duration=30):
    y, sr = librosa.load(audio_path, sr=sr, duration=duration)
    y_harmonic = librosa.effects.harmonic(y)

    feature_dict = {}

    chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr)
    chroma_cqt = librosa.feature.chroma_cqt(y=y, sr=sr)
    chroma_cens = librosa.feature.chroma_cens(y=y, sr=sr)

    feature_dict.update(compute_summary_stats(chroma_stft, "chroma_stft"))
    feature_dict.update(compute_summary_stats(chroma_cqt, "chroma_cqt"))
    feature_dict.update(compute_summary_stats(chroma_cens, "chroma_cens"))

    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
    feature_dict.update(compute_summary_stats(mfcc, "mfcc"))

    rmse = librosa.feature.rms(y=y)
    feature_dict.update(compute_summary_stats(rmse, "rmse"))

    spectral_centroid = librosa.feature.spectral_centroid(y=y, sr=sr)
    spectral_bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    spectral_contrast = librosa.feature.spectral_contrast(y=y, sr=sr)
    spectral_rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)

    feature_dict.update(compute_summary_stats(spectral_centroid,
"spectral_centroid"))
    feature_dict.update(compute_summary_stats(spectral_bandwidth,
"spectral_bandwidth"))
    feature_dict.update(compute_summary_stats(spectral_contrast,
"spectral_contrast"))
    feature_dict.update(compute_summary_stats(spectral_rolloff,
"spectral_rolloff"))

    tonnetz = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    feature_dict.update(compute_summary_stats(tonnetz, "tonnetz"))

    zcr = librosa.feature.zero_crossing_rate(y)
    feature_dict.update(compute_summary_stats(zcr, "zcr"))

    feature_df = pd.DataFrame([feature_dict])
    feature_df = feature_df.reindex(columns=structured_feature_columns,

```

```

fill_value=0.0)
    feature_df = feature_df.replace([np.inf, -np.inf], np.nan).fillna(0.0)

    return feature_df

def extract_mel_spectrogram(audio_path, sr=22050, duration=30, n_mels=128,
target_width=1292):
    y, sr = librosa.load(audio_path, sr=sr, duration=duration)

    mel_spec = librosa.feature.melspectrogram(
        y=y,
        sr=sr,
        n_mels=n_mels
    )

    mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)

    if mel_spec_db.shape[1] < target_width:
        pad_width = target_width - mel_spec_db.shape[1]
        mel_spec_db = np.pad(
            mel_spec_db,
            ((0, 0), (0, pad_width)),
            mode="constant"
        )
    else:
        mel_spec_db = mel_spec_db[:, :target_width]

    return mel_spec_db

def predict_genre_10genre(audio_path, structured_weight=0.4, cnn_weight=0.6):
    structured_features = extract_structured_features(audio_path)
    structured_features_scaled = structured_scaler.transform(structured_features)
    structured_probs = structured_model.predict_proba(structured_features_scaled)
[0]

    mel_spec = extract_mel_spectrogram(audio_path)
    cnn_input = mel_spec[np.newaxis, ..., np.newaxis].astype(np.float32)

    denom = np.max(np.abs(cnn_input))
    if denom != 0:
        cnn_input = cnn_input / denom

    cnn_probs = cnn_model.predict(cnn_input, verbose=0)[0]

    hybrid_probs = (structured_weight * structured_probs) + (cnn_weight *
cnn_probs)

    predicted_index = int(np.argmax(hybrid_probs))
    predicted_genre = class_names[predicted_index]

    top3_idx = np.argsort(hybrid_probs)[::-1][:3]
    top3_results = [(class_names[i], float(hybrid_probs[i])) for i in top3_idx]

    return {
        "predicted_genre": predicted_genre,
        "top3_predictions": top3_results,
        "structured_probs": structured_probs,

```

```
"cnn_probs": cnn_probs,
"hybrid_probs": hybrid_probs
}
```

In [5]:

```
# =====
# 5. BUILD A SMALL DEMO SAMPLE
# =====
# We take a few test examples per genre so the demo stays manageable.

SEED = 42
SAMPLES_PER_GENRE = 3

test_df = pilot_df[pilot_df["split"] == "test"].copy()

demo_parts = []

for genre in sorted(test_df["genre_top"].unique()):
    genre_df = test_df[test_df["genre_top"] == genre].copy()
    sampled_df = genre_df.sample(
        n=min(SAMPLES_PER_GENRE, len(genre_df)),
        random_state=SEED
    )
    demo_parts.append(sampled_df)

demo_df = pd.concat(demo_parts).reset_index(drop=True)

print("Demo sample shape:", demo_df.shape)
display(demo_df.head(10))
```

Demo sample shape: (30, 6)

	track_id	genre_top	split	audio_path	audio_exists	can_load
0	34326	Classical	test	../data/raw/audio/fma_large\034\034326.mp3	True	True
1	36469	Classical	test	../data/raw/audio/fma_large\036\036469.mp3	True	True
2	29661	Classical	test	../data/raw/audio/fma_large\029\029661.mp3	True	True
3	119147	Electronic	test	../data/raw/audio/fma_large\119\119147.mp3	True	True
4	4736	Electronic	test	../data/raw/audio/fma_large\004\004736.mp3	True	True
5	31061	Electronic	test	../data/raw/audio/fma_large\031\031061.mp3	True	True
6	118412	Experimental	test	../data/raw/audio/fma_large\118\118412.mp3	True	True
7	99991	Experimental	test	../data/raw/audio/fma_large\099\099991.mp3	True	True
8	10526	Experimental	test	../data/raw/audio/fma_large\010\010526.mp3	True	True
9	116659	Folk	test	../data/raw/audio/fma_large\116\116659.mp3	True	True



In [6]:

```

# =====
# 6. RUN BATCH PREDICTIONS
# =====

results_rows = []

for _, row in demo_df.iterrows():
    audio_path = row["audio_path"]
    actual_genre = row["genre_top"]
    track_id = row["track_id"]

    try:
        result = predict_genre_10genre(
            audio_path,
            structured_weight=STRUCTURED_WEIGHT,
            cnn_weight=CNN_WEIGHT
        )

        top3 = result["top3_predictions"]

        results_rows.append({
            "track_id": track_id,
            "actual_genre": actual_genre,
            "predicted_genre": result["predicted_genre"],
            "top1_probability": top3[0][1],
            "top2_genre": top3[1][0],
            "top2_probability": top3[1][1],
            "top3_genre": top3[2][0],
            "top3_probability": top3[2][1],
            "correct_top1": actual_genre == result["predicted_genre"]
        })

    except Exception as e:
        results_rows.append({
            "track_id": track_id,
            "actual_genre": actual_genre,
            "predicted_genre": f"ERROR: {e}",
            "top1_probability": np.nan,
            "top2_genre": np.nan,
            "top2_probability": np.nan,
            "top3_genre": np.nan,
            "top3_probability": np.nan,
            "correct_top1": False
        })

batch_results_df = pd.DataFrame(results_rows)

print("Batch inference results:")
display(batch_results_df)

```

Batch inference results:

	track_id	actual_genre	predicted_genre	top1_probability	top2_genre	top2_probability	
0	34326	Classical	Experimental	0.183276	Instrumental	0.175362	
1	36469	Classical	Spoken	0.214001	Experimental	0.164517	li
2	29661	Classical	Spoken	0.224694	Experimental	0.199482	li
3	119147	Electronic	Instrumental	0.217108	Spoken	0.189381	
4	4736	Electronic	Experimental	0.218835	Rock	0.190993	
5	31061	Electronic	Spoken	0.210415	Electronic	0.202559	E
6	118412	Experimental	Electronic	0.216893	Experimental	0.180265	
7	99991	Experimental	Experimental	0.216027	Rock	0.179807	
8	10526	Experimental	Pop	0.247233	Electronic	0.198665	E
9	116659	Folk	Spoken	0.225791	Experimental	0.175733	li
10	42471	Folk	Experimental	0.183627	Electronic	0.177442	
11	154624	Folk	Experimental	0.193914	Spoken	0.186837	
12	75088	Hip-Hop	Electronic	0.199683	Experimental	0.181129	
13	110750	Hip-Hop	Electronic	0.254528	Experimental	0.179010	li
14	110745	Hip-Hop	Pop	0.202648	Experimental	0.187827	
15	115174	Instrumental	Experimental	0.179886	Rock	0.171255	li
16	86348	Instrumental	Electronic	0.176644	Pop	0.166562	E
17	100851	Instrumental	Experimental	0.192187	Electronic	0.164527	li
18	98312	International	Experimental	0.200710	Electronic	0.195925	li
19	121920	International	Electronic	0.200798	Experimental	0.191002	
20	29357	International	Experimental	0.200831	Electronic	0.187911	
21	150012	Pop	Spoken	0.173734	Instrumental	0.167404	
22	18588	Pop	Instrumental	0.199727	Experimental	0.189421	
23	114660	Pop	Electronic	0.173954	Pop	0.161010	E
24	152353	Rock	Experimental	0.210071	Instrumental	0.177102	
25	109440	Rock	Experimental	0.225493	Electronic	0.193512	
26	61800	Rock	Spoken	0.207595	Experimental	0.186540	li
27	33984	Spoken	Spoken	0.320554	Experimental	0.170502	
28	44411	Spoken	Spoken	0.352699	Experimental	0.188249	li
29	125595	Spoken	Spoken	0.267665	Experimental	0.183200	

In [7]:

```
# =====
# 7. SUMMARY METRICS FOR THE DEMO SAMPLE
# =====

demo_accuracy = batch_results_df["correct_top1"].mean()

print("Demo top-1 accuracy on sampled files:", demo_accuracy)

summary_by_actual = (
    batch_results_df
    .groupby("actual_genre")["correct_top1"]
    .agg(["count", "sum", "mean"])
    .rename(columns={
        "count": "samples",
        "sum": "correct_predictions",
        "mean": "accuracy"
    })
)

print("\nAccuracy by actual genre:")
display(summary_by_actual)
```

Demo top-1 accuracy on sampled files: 0.13333333333333333

Accuracy by actual genre:

	samples	correct_predictions	accuracy
actual_genre			
Classical	3	0	0.000000
Electronic	3	0	0.000000
Experimental	3	1	0.333333
Folk	3	0	0.000000
Hip-Hop	3	0	0.000000
Instrumental	3	0	0.000000
International	3	0	0.000000
Pop	3	0	0.000000
Rock	3	0	0.000000
Spoken	3	3	1.000000

In [8]:

```
# =====
# 8. SAVE DEMO RESULTS
# =====
```

```
os.makedirs("../data/processed", exist_ok=True)

batch_results_df.to_csv(
    "../data/processed/batch_inference_demo_10genre.csv",
    index=False
)

summary_by_actual.to_csv(
    "../data/processed/batch_inference_demo_10genre_summary.csv"
)

print("Saved batch inference demo results.")
```

Saved batch inference demo results.

In [9]:

```
# =====
# 9. INTERPRETATION NOTES
# =====

print("1. This notebook applies the final 10-genre hybrid system to multiple
readable audio files.")
print("2. It shows how the deployed pipeline behaves beyond a single example.")
print("3. The exported tables can be used in the report or presentation as
qualitative evidence.")
print("4. This notebook is intended as a demonstration notebook, not as a
replacement for the formal test-set evaluation notebooks.")
```

1. This notebook applies the final 10-genre hybrid system to multiple readable audio files.
2. It shows how the deployed pipeline behaves beyond a single example.
3. The exported tables can be used in the report or presentation as qualitative evidence.
4. This notebook is intended as a demonstration notebook, not as a replacement for the formal test-set evaluation notebooks.